

**COE180.1**

# Introduction to Hardware Description Language

Lab Manual (2025)

---

Name & Student number



# Contents

|          |                                                        |          |
|----------|--------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction to HDL using Verilog - Logic Gates</b> | <b>3</b> |
| 1.1      | Objectives . . . . .                                   | 3        |
| 1.2      | Introduction . . . . .                                 | 3        |
| 1.3      | Background: Core Verilog Concepts . . . . .            | 4        |
| 1.4      | Verification: The Role of the Testbench . . . . .      | 5        |
| 1.5      | Simulation . . . . .                                   | 8        |
| 1.6      | Equipment . . . . .                                    | 9        |
| 1.7      | Lab Tasks and Procedure . . . . .                      | 9        |
| 1.8      | Gates Truth Tables . . . . .                           | 10       |
| 1.9      | Pre-lab Questions . . . . .                            | 11       |
| 1.10     | Post-lab Questions . . . . .                           | 11       |
| 1.11     | Lab Report Requirements . . . . .                      | 11       |
| 1.12     | Grading . . . . .                                      | 11       |

## A Guide to the Notes in this Manual

Throughout this lab manual, you will find specially marked boxes designed to enhance your learning experience. These notes provide context, highlight critical concepts, and offer insights that go beyond the basic procedures. Pay close attention to them.

### Notable Information

This box contains tips, guidance, and high-level context that a professor might share during a lecture to help you connect ideas or prepare for the lab.

### Important Note for Students

This box highlights crucial concepts, common industry practices, or potential pitfalls. Understanding the information in these notes is essential for building a strong foundation in digital design.

### Hardware Inference

This box explains what physical hardware a synthesis tool is likely to create from your Verilog code. It helps bridge the gap between the HDL you write and the physical circuit you are actually designing.

# 1 Introduction to HDL using Verilog - Logic Gates

## 1.1 Objectives

By the end of this lab, you will be able to:

- Gain a solid understanding of the basic structure of Verilog modules and ports.
- Be able to implement simple digital logic circuits using Verilog HDL, employing different modeling styles.
- Become familiar with the simulation and verification process in HDL design using Icarus Verilog and testbenches.
- Understand the trade-offs between different modeling approaches in Verilog and choose the most appropriate style for a given design scenario.
- Develop the skills to write basic testbenches to provide input stimuli and observe the outputs of your designs.

## 1.2 Introduction

Digital circuits are the foundation of modern electronics, built from fundamental components called **logic gates**. These gates perform basic logical operations on binary inputs to produce binary outputs. Designing and implementing these circuits efficiently requires a systematic approach, which is provided by **Hardware Description Languages (HDLs)**.

Hardware Description Languages (HDLs) like Verilog are essential tools for designing and implementing digital circuits. In this lab, we'll take our first steps into the world of Verilog by creating models for fundamental logic gates. We'll explore different modeling styles (dataflow, behavioral, and structural) to understand better how Verilog can be used to represent digital logic.

### 1.2.1 Practical Applications

While logic gates are fundamental building blocks, they form the foundation of virtually all digital systems. Understanding their implementation in HDL is crucial for:

- **Integrated Circuit Design:** The gates you design in this lab are the same basic components used to create complex microprocessors, memory chips, and application-specific integrated circuits (ASICs).
- **FPGA Programming:** Field-Programmable Gate Arrays contain configurable logic blocks that are programmed using HDL descriptions of gates and more complex circuits.
- **Digital System Prototyping:** Before committing to expensive hardware manufacturing, engineers use HDL simulations to verify that their gate-level designs function correctly.
- **Embedded Systems:** Many embedded controllers include programmable logic sections that are configured using gate-level descriptions.
- **Computer Architecture:** Understanding how to build complex circuits from basic gates is essential for designing processors, memory systems, and peripheral controllers.

### Verilog HDL

Verilog HDL is a powerful language used to describe the structure and behavior of digital circuits. It allows designers to express complex digital systems in a textual format that can be simulated, synthesized, and ultimately implemented on hardware. In this lab, we will embark on our journey into the world of Verilog by focusing on implementing basic logic gates.

## 1.3 Background: Core Verilog Concepts

### 1.3.1 The Verilog Module

The fundamental building block in Verilog is the **module**. It is a self-contained design with inputs and outputs, analogous to a schematic for an integrated circuit. All your designs will be encapsulated within a `module...endmodule` block.

### 1.3.2 Modeling Styles in Verilog

Verilog allows you to describe hardware at different levels of abstraction. We will explore three primary styles for combinational logic:

- **Dataflow:** Describes the flow of data through the circuit using continuous assignments (**assign**) and Boolean expressions. This style is concise and intuitive for simple combinational logic.
- **Behavioral:** Describes the circuit's behavior algorithmically using procedural statements like **always**, **if-else**, and **case**. This style is powerful for modeling complex logic.
- **Structural:** Describes a circuit by instantiating and interconnecting lower-level components. This promotes hierarchical design and code reuse.

#### Notable Information

The choice of style often depends on the task. Use Dataflow for simple equations, Behavioral for complex decision logic (like a state machine's logic), and Structural for connecting well-defined modules into a larger system. For this lab, you will practice all three to understand their differences.

### Modeling Style Examples: A 2-Input AND Gate

- AND Gate

| $p$ | $q$ | $p \wedge q$ |
|-----|-----|--------------|
| $T$ | $T$ | $T$          |
| $T$ | $F$ | $F$          |
| $F$ | $T$ | $F$          |
| $F$ | $F$ | $F$          |

**Dataflow Style:** Uses the **assign** statement.

```
module and_gate_df (
    input a,
    input b,
    output y
);
    assign y = a & b;
endmodule
```

#### Hardware Inference

This code describes a simple combinational logic function. A synthesis tool will infer a single **AND gate** from the expression `a & b`. The **assign** keyword continuously drives the output `y` whenever the inputs `a` or `b` change. This directly models the behavior of a physical AND gate.

**Behavioral Style:** Uses an `always` block. The output must be declared as `reg`.

```
module and_gate_bh (  
    input a,  
    input b,  
    output y  
);  
    always @(*) begin  
        y = a & b;  
    end  
endmodule
```

#### Hardware Inference

The `always @(*)` block tells the synthesis tool that this block models combinational logic. The tool analyzes the function inside the block (`y = a & b`) and will generate the exact same hardware as the dataflow style: a single **AND gate**. The `@(*)` is a sensitivity list that ensures the block executes whenever any input (a or b) changes, which is required for correct combinational logic.

**Structural Style:** Builds a circuit by instantiating other modules.

Structural: NAND Gate (using AND and NOT gates):

```
module nand_gate_st (  
    input a,  
    input b,  
    output y  
);  
    wire and_out;  
  
    and_gate and1 (a, b, and_out);  
    not_gate not1 (and_out, y);  
endmodule
```

#### Hardware Inference

This code describes a specific hardware structure. A synthesis tool will not generate a single gate but will instead create the network of components you specified: it will take the pre-designed layouts for an **AND gate** and a **NOT gate** and connect the output of the AND gate to the input of the NOT gate. The final hardware is a distinct **NAND gate** built from these two components, just as if you had drawn this schematic by hand.

## 1.4 Verification: The Role of the Testbench

A design is incomplete until it has been verified. A **testbench** is a non-synthesizable Verilog module written for the sole purpose of testing another module, which we call the **Design Under Test (DUT)**. Testbenches are crucial for verifying the correctness of our logic, acting as standalone modules that stimulate the DUT with various inputs and allow us to monitor its outputs. This systematic approach to testing ensures our gates operate as expected according to their truth tables.

Key characteristics of a testbench:

- **It is for simulation only:** Testbench code is not intended to be converted into physical hardware (it is non-synthesizable). It uses special features and commands that only work within a simulator.
- **It has no external inputs or outputs:** The testbench is the top-level module in a simulation. It generates all the stimuli internally and checks the DUT's responses internally.
- **It controls time:** The testbench defines the sequence of events and how much simulation time passes between them.

## Why are Testbenches Essential?

- 1. Automated Verification:** A testbench can apply all input combinations systematically and automatically.
- 2. Finding Bugs Early:** It is vastly cheaper and faster to find a logic error in simulation than on physical hardware.
- 3. Regression Testing:** After modifying a design, you can re-run its testbench to ensure you haven't broken anything.

### Notable Information

#### Why Verification is Critical

In digital design, a common mantra is: **"A design is only as good as its testbench."** Verification is not an afterthought; it is a critical part of the design process. It is vastly cheaper and faster to find a logic error in simulation than it is to debug a malfunctioning circuit on physical hardware. A well-written testbench automates this process, allowing you to test every possible input, including tricky corner cases.

## Anatomy of a Testbench: A Step-by-Step Breakdown

To understand how a testbench works, let's analyze a complete and correct example for the AND gate we designed earlier.

```
// 1. Timescale Directive
`timescale 1ns / 1ps

// 2. Module Declaration (no ports)
module and_gate_tb;

    // 3. Signal Declarations
    reg a, b; // Inputs to DUT are 'reg'
    wire y;   // Outputs from DUT are 'wire'

    // 4. Instantiate the Design Under Test (DUT)
    and_gate_df dut (
        .a(a),
        .b(b),
        .y(y)
    );

    // 5. Stimulus and Verification Block
    initial begin
        // A. Setup waveform dumping for GTKWave
        $dumpfile("and_gate_tb.vcd");
        $dumpvars(0, and_gate_tb);

        // B. (Optional) Monitor console output
        $monitor("Time=%0t ns | a=%b b=%b | y=%b", $time, a, b, y);

        // C. Apply test vectors (stimuli)
        a = 0; b = 0; #10; // Wait 10ns
        a = 0; b = 1; #10;
        a = 1; b = 0; #10;
        a = 1; b = 1; #10;

        // D. End the simulation
        $finish;
    end
endmodule
```

1. **Timescale Directive** The `timescale 1ns / 1ps` directive defines the simulation's sense of time.
  - **1ns**: This is the **time unit**. When you write `#10`, it means "wait for 10 nanoseconds."
  - **1ps**: This is the **time precision**, the smallest time step the simulator can resolve.
2. **Module Declaration** The testbench module, `and_gate_tb`, has no ports. It is a self-contained, top-level environment.
3. **Signal Declarations (reg vs. wire)** This is a critical rule for all testbenches:
  - **reg a, b**:: Signals that drive the DUT's inputs **must** be declared as **reg**. This is because they are assigned values procedurally (inside the `initial` block).
  - **wire y**:: Signals connected to the DUT's outputs **must** be declared as **wire**. They are passively driven by the DUT, and the testbench simply "listens" to them.
4. **Instantiating the DUT** This line creates an instance of your `and_gate_df` module inside the testbench. We use named port connections (`.a(a)`) which is a safer practice than connecting by position, as it avoids errors if the port order changes.
5. **The initial Block** This procedural block starts at time 0 and executes its statements sequentially. It contains the entire test plan.
  - A. **Waveform Dumping**: The `$dumpfile` command specifies the output filename for the waveform viewer. `$dumpvars(0, and_gate_tb)` tells the simulator to record all signal value changes within the testbench module for viewing in GTKWave.
  - B. **Monitoring**: The `$monitor` system task is a handy debugging tool. It prints to the console whenever any of its listed signals change, giving you a live text-based view of the simulation.
  - C. **Applying Stimuli**: This is the core of the test. We assign values to our **reg** signals (`a` and `b`) and use delays (`#10`) to wait for a specific amount of simulation time before applying the next set of inputs. This creates the sequence of test vectors.
  - D. **Finishing**: The `$finish` command tells the simulator to terminate once all test vectors have been applied.

#### Another version of AND Gate Testbench: From our Lecture

```

`timescale 1ns / 1ps

module and_gate_tb;
    // Inputs to the DUT are declared as 'reg'
    reg a, b, clk;
    // Outputs from the DUT are declared as 'wire'
    wire y;

    // Instantiate the Design Under Test (DUT)
    and_gate_bh dut (.a(a), .b(b), .y(y));

    // Generate clock signal
    always #5 clk = ~clk;

    // Procedural block to define the stimuli
    initial begin
        clk = 0; //initialize clk
        // Setup waveform dumping
        $dumpfile("and_gate_tb.vcd");
        $dumpvars(0, and_gate_tb);

        // Apply all possible inputs

```

```

        a = 0; b = 0; #10; // Wait 10 time units
        a = 0; b = 1; #10;
        a = 1; b = 0; #10;
        a = 1; b = 1; #10;

    $finish; // End the simulation
end

    // Monitor and verify outputs
    always @(*) begin
        $display("a=%b, b=%b, y=%b", a, b, y);
    end
endmodule

```

- `$dumpfile("and_gate_tb.vcd")`: This line specifies the name of the VCD file that will be created to store the signal values during simulation.
- `$dumpvars(0, and_gate_tb)`: This line tells the simulator to start dumping the values of all variables within the `and_gate_tb` module (and its submodules if any) into the VCD file. The 0 indicates hierarchical dumping, meaning it will include signals from any instantiated modules within the testbench.

## 1.5 Simulation

To ensure the correctness of our Verilog designs, we will utilize Icarus Verilog, an open-source Verilog compiler and simulator. It will allow us to compile our Verilog code into a format suitable for simulation and then execute the simulation to observe the behavior of our circuits.

Finally, we will use GTKWave, a waveform viewer, to visualize the simulation results and gain a deeper understanding of the dynamic behavior of our circuits.

- Simulate and Verify using Icarus Verilog + GTKWave

1. Save your Verilog modules and testbenches with a `.v` extension.
2. **Compile** the design and testbench. The `-o` flag specifies the output executable name.

```

iverilog -o testbench_filename testbench_filename.v instantiated_module_filename.v

iverilog -o and_gate_tb and_gate_tb.v and_gate_bh.v

```

3. **Run the simulation** using the `vvp` command. This executes the compiled output and generates the VCD file.

```

vvp testbench_filename

vvp and_gate_tb

```

This line will simulate, generating a `'testbench_filename.vcd'` file.

Observe the simulation output in the console to ensure the logic gates function correctly according to their truth tables for all descriptions.

4. **View the waveforms** in GTKWave to visually verify the timing and logic.

```

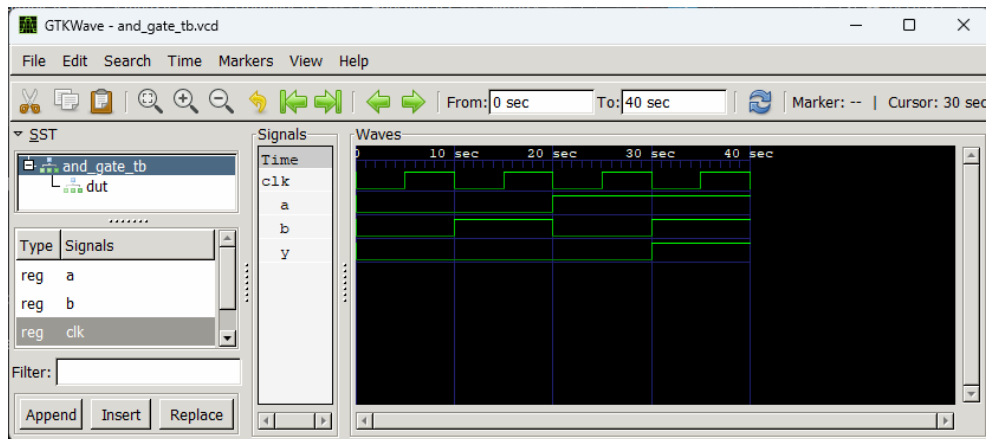
gtkwave testbench_filename.vcd

gtkwave and_gate_tb.vcd

```

- In GTKWave:

- Select signals from the **SST** pane and drag them to the viewer.



**Figure 1.1:** GTKWave waveform of AND gate

- Use the mouse wheel to zoom and pan.
- Use the mouse to measure time differences between events.
- Verify the output waveforms against the expected truth tables.
- ◻ Observe the waveforms to ensure the logic gates function correctly according to their truth tables for all descriptions.

By incorporating GTKWave into your workflow, you can better understand how your Verilog designs behave over time. This visual representation is invaluable for debugging and verifying the functionality of your circuits.

## 1.6 Equipment

Computer with Icarus Verilog simulator installed

## 1.7 Lab Tasks and Procedure

### 1. Familiarize Yourself with Verilog Syntax

**[a]** Review the fundamental Verilog concepts:

- Modules and Ports
- Data Types: **wire** vs. **reg** (see Important Note)
- Operators (& (AND), | (OR), ~ (NOT), ^ (XOR))
- Assign Statements
- Always Blocks
- Case Statements

#### Important Note for Students

The keyword **reg** does **not** always infer a physical register (flip-flop). It is a variable that can hold a value and is assigned to inside **always** or **initial** blocks. A **wire** is a net type and is typically driven by a module output or an **assign** statement. Confusing these is a common source of errors.

**2. Implement Logic Gates:** For each of the following gates, create three separate Verilog files—one for each modeling style (Dataflow, Behavioral, and Structural).

- OR, NOT, NAND, NOR, XOR, XNOR (See Section 1.8 for the truth tables)

**3. Write a Testbench for Each Gate:** Create a testbench for each logic gate implementation. The testbench should cycle through all possible input combinations to verify the gate's truth table.

**4. Simulate and Analyze:** Use the Icarus Verilog and GTKWave toolchain to verify your designs.

**[a] Compile** the design and its testbench. The `-o` flag specifies the output executable name.

```
iverilog -o my_simulation design_file.v testbench_file.v
```

**[b] Run** the simulation using the `vvp` command. This will generate the VCD waveform file.

```
vvp my_simulation
```

**[c] View** the waveforms in GTKWave to visually verify the logic and timing.

```
gtkwave your_dumpfile.vcd
```

## 1.8 Gates Truth Tables

### ■ OR Gate

| $p$ | $q$ | $p \vee q$ |
|-----|-----|------------|
| $T$ | $T$ | $T$        |
| $T$ | $F$ | $T$        |
| $F$ | $T$ | $T$        |
| $F$ | $F$ | $F$        |

### ■ NOT Gate

| $p$ | $\neg p$ |
|-----|----------|
| $T$ | $F$      |
| $F$ | $T$      |

### ■ NAND Gate

| $p$ | $q$ | $\neg(p \wedge q)$ |
|-----|-----|--------------------|
| $T$ | $T$ | $F$                |
| $T$ | $F$ | $T$                |
| $F$ | $T$ | $T$                |
| $F$ | $F$ | $T$                |

### ■ NOR Gate

| $p$ | $q$ | $\neg(p \vee q)$ |
|-----|-----|------------------|
| $T$ | $T$ | $F$              |
| $T$ | $F$ | $F$              |
| $F$ | $T$ | $F$              |
| $F$ | $F$ | $T$              |

### ■ XOR Gate

| $p$ | $q$ | $(\neg p \wedge q) \vee (p \wedge \neg q)$ |
|-----|-----|--------------------------------------------|
| $T$ | $T$ | $F$                                        |
| $T$ | $F$ | $T$                                        |
| $F$ | $T$ | $T$                                        |
| $F$ | $F$ | $F$                                        |

### ■ XNOR Gate

| $p$ | $q$ | $(\neg p \wedge \neg q) \vee (p \wedge q)$ |
|-----|-----|--------------------------------------------|
| $T$ | $T$ | $T$                                        |
| $T$ | $F$ | $F$                                        |
| $F$ | $T$ | $F$                                        |
| $F$ | $F$ | $T$                                        |

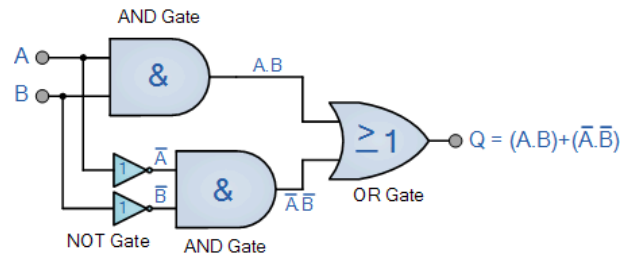


Figure 1.2: XNOR Gate

## 1.9 Pre-lab Questions

1. What is a truth table?
2. Which gates are called universal gates and why?
3. What is the fundamental difference between an HDL (like Verilog) and a software programming language (like C)?
4. Define "identifier" in the context of Verilog.
5. A 2-input logic circuit has one HIGH and one LOW input, and its output is HIGH. What type of gate is it?
6. A logic circuit requires all its inputs to be HIGH to make the output HIGH. What type of gate is it?

## 1.10 Post-lab Questions

1. What is meant by "ports" in a Verilog module?
2. List the different port modes (e.g., input, output) and their purposes.
3. Describe the different categories of operators in Verilog (e.g., bitwise, logical).
4. What is the difference between blocking (=) and non-blocking (<=) assignment operators? (Note: We will cover this in detail for sequential logic, but research the basic difference).
5. What is meant by "simulation" in the context of digital design?

## 1.11 Lab Report Requirements

Submit a comprehensive lab report containing:

- Answers to all pre-lab and post-lab questions.
- Complete, well-commented Verilog code for all logic gates and their testbenches.
- Screenshots of the output signal waveforms from GTKWave for each gate, with a brief analysis confirming correct operation.

## 1.12 Grading

| Grading Criteria | Points    |
|------------------|-----------|
| Pre-labWork      | 15 points |
| Lab Performance  | 35 points |
| Post-lab Work    | 20 points |
| Lab report       | 30 points |

For the lab performance - at a minimum, demonstrate the operation of all the logic circuits

The lab report will be graded as follows (for the 30 points):

- VERILOG code for each experiments 15 points
- Output signal waveform for all experiments 15 points